

24. Strategy Design Pattern in Java

24.1 Introduction

The **Strategy Pattern** is a **behavioral pattern** that enables selecting an algorithm's behavior at runtime.

24.2 Problem it Solves

Removes conditional logic by encapsulating algorithms (strategies) in separate classes and makes them interchangeable.

24.3 What is Strategy Pattern?

- Defines a family of algorithms
- Encapsulates each one in a separate class
- Makes them interchangeable at runtime

24.4 Real-World Analogy

Different **navigation strategies** in Google Maps: driving, walking, or cycling.

24.5 Structure

```
package com.mannemlokes;

public interface Strategy {
    void execute();
}

class Context {
    private Strategy strategy;

    public Context(Strategy strategy) {
        this.strategy = strategy;
    }

    public void setStrategy(Strategy strategy) {
        this.strategy = strategy;
    }

    public void perform() {
        strategy.execute();
    }
}
```

24.6 Benefits

- Avoids large if-else blocks
- Easy to add new strategies
- Promotes Open/Closed Principle

24.7 Implementation Steps

1. Create a Strategy interface
2. Implement multiple concrete strategies

3. Use Context to switch between them at runtime

24.8 Examples

Example1: Payment Strategies

```
package com.mannemlokes.example1;

interface PaymentStrategy {
    void pay(int amount);
}

class CreditCardPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Paid $" + amount + " using Credit Card.");
    }
}

class PayPalPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Paid $" + amount + " using PayPal.");
    }
}

class ShoppingCart {
    private PaymentStrategy strategy;

    public void setPaymentStrategy(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    public void checkout(int amount) {
        strategy.pay(amount);
    }
}

public class Example1StrategyClient {
    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart();

        cart.setPaymentStrategy(new CreditCardPayment());
        cart.checkout(500);

        cart.setPaymentStrategy(new PayPalPayment());
        cart.checkout(300);
    }
}
```

Output:

```
Paid $500 using Credit Card.
Paid $300 using PayPal.
```

Example2: Compression Strategies

```
package com.mannemlokes.example2;

interface CompressionStrategy {
    void compress(String file);
}
```

```

}

class ZipCompression implements CompressionStrategy {
    public void compress(String file) {
        System.out.println("Compressing " + file + " as ZIP.");
    }
}

class RarCompression implements CompressionStrategy {
    public void compress(String file) {
        System.out.println("Compressing " + file + " as RAR.");
    }
}

class Compressor {
    private CompressionStrategy strategy;

    public Compressor(CompressionStrategy strategy) {
        this.strategy = strategy;
    }

    public void setStrategy(CompressionStrategy strategy) {
        this.strategy = strategy;
    }

    public void compressFile(String file) {
        strategy.compress(file);
    }
}

public class Example2CompressionClient {
    public static void main(String[] args) {
        Compressor compressor = new Compressor(new ZipCompression());
        compressor.compressFile("report.txt");

        compressor.setStrategy(new RarCompression());
        compressor.compressFile("presentation.ppt");
    }
}

```

Output:

```

Compressing report.txt as ZIP.
Compressing presentation.ppt as RAR.

```

Example3: AI Behavior Strategy in Game

```

package com.mannemlokes.example3;

interface AIStrategy {
    void takeTurn();
}

class AggressiveStrategy implements AIStrategy {
    public void takeTurn() {
        System.out.println("AI attacks the enemy directly!");
    }
}

```

```

class DefensiveStrategy implements AIStrategy {
    public void takeTurn() {
        System.out.println("AI builds defenses.");
    }
}

class AIContext {
    private AIStrategy strategy;

    public AIContext(AIStrategy strategy) {
        this.strategy = strategy;
    }

    public void changeStrategy(AIStrategy strategy) {
        this.strategy = strategy;
    }

    public void executeTurn() {
        strategy.takeTurn();
    }
}

public class Example3GameClient {
    public static void main(String[] args) {
        AIContext ai = new AIContext(new AggressiveStrategy());
        ai.executeTurn(); // Aggressive

        ai.changeStrategy(new DefensiveStrategy());
        ai.executeTurn(); // Defensive
    }
}

```

Output:

```

AI attacks the enemy directly!
AI builds defenses.

```

24.9 Strategy vs State vs Template

Pattern	Purpose
Strategy	Choose algorithm at runtime
State	Change behavior based on state
Template	Define skeleton and allow steps override

24.10 Pros and Cons

✓ Pros

- Promotes flexibility and extensibility
- Avoids code duplication for similar operations
- Runtime behavior changes possible

✗ Cons

- Increased number of classes

- Client must be aware of different strategies

24.11 When to Use / Not to Use

Use When:

- Multiple variations of an algorithm exist
- You want to switch behavior dynamically

Avoid When:

- Algorithm doesn't vary
- Few conditions (use simple if-else)

24.12 Summary

Feature	Description
Intent	Encapsulate interchangeable behavior
Use Case	Payment, compression, sorting, AI
Key Benefit	Swap algorithms without modifying code

The **Strategy Pattern** is ideal when you have multiple interchangeable behaviors and want to keep code flexible and clean.